SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	M. Theshon Mishmaan	**Reg. No.:**	19257106
Section / Batch:	CSE-AI	**Date:**	04-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

2. Evaluate the scalability of Merge Sort in parallel computing environments by simulating multi-threaded execution. Record execution time for different numbers of threads. Analyze speedup achieved through parallelism. Interpret results and justify the suitability of Merge Sort for parallel processing.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				

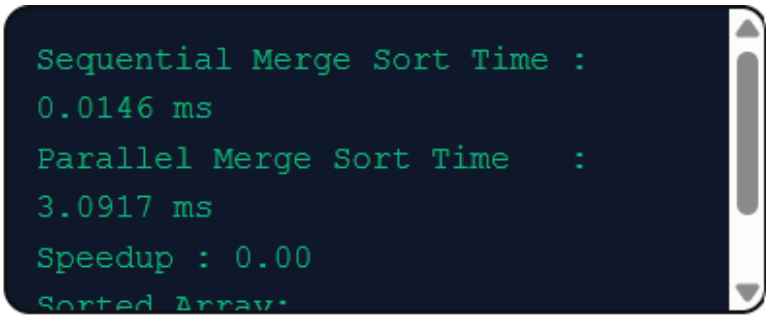
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Code:

```java
import java.util.*;
import java.util.concurrent.*;

public class Main {

    // Sequential Merge Sort
    static void mergeSort(int[] arr, int l, int r) {
        if (l >= r)
            return;

        int mid = (l + r) / 2;

        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);

        merge(arr, l, mid, r);
    }

    static void merge(int[] arr, int l, int mid, int r) {

        int n1 = mid - l + 1;
        int n2 = r - mid;

        int[] left = new int[n1];
        int[] right = new int[n2];

        for (int i = 0; i < n1; i++)
            left[i] = arr[l + i];

        for (int i = 0; i < n2; i++)
            right[i] = arr[mid + 1 + i];

        int i = 0, j = 0, k = l;
```

Output:


Sequential Merge Sort Time :
0.0146 ms
Parallel Merge Sort Time :
3.0917 ms
Speedup : 0.00
Sorted Array: